



Instituto Politécnico Nacional

Semestre A26

Algoritmos



Alumno: **González Mendoza Juan Pablo**

Fecha: **Junio, 2026**

Tarea 3 Práctica PILAS.

Cuando un programa ejecuta una función, el sistema operativo y el compilador colaboran para administrar la memoria mediante una estructura conocida como pila de llamadas (*call stack*) [1]. Esta pila opera bajo el principio LIFO (*Last-In, First-Out*): la última función invocada es la primera en completarse y liberar su espacio [2]. Cada vez que se llama a una función se crea un marco de pila (*stack frame*) o registro de activación que almacena la dirección de retorno, los parámetros recibidos y las variables locales [2, 3]. Comprender cómo funciona esta estructura es fundamental para el análisis de algoritmos, ya que la profundidad de la pila afecta directamente la viabilidad de soluciones recursivas y el consumo de memoria. En esta práctica se implementan cinco experimentos en C++ que permiten observar y verificar el comportamiento de la pila de funciones: el orden de las variables, la relación entre marcos de distintas funciones, la alteración indirecta de valores, los límites de profundidad y la diferencia entre paso por valor y paso por referencia.

Marco Teórico

En cuanto al marco teórico, un marco de pila típico contiene, de la base hacia el tope, los siguientes elementos: la dirección de retorno (la instrucción a la que el procesador debe regresar al terminar la función), el puntero de marco anterior o *saved frame pointer* (enlace al marco de la función llamadora), los parámetros (copias o referencias de los argumentos recibidos) y las variables locales (espacio para los datos temporales de la función). En la mayoría de las arquitecturas x86-64, la pila crece hacia direcciones de memoria más bajas [2], sin embargo el compilador puede reorganizar variables dentro de un mismo marco por razones de alineamiento u optimización.

Sobre los límites de la pila, el tamaño máximo está determinado por el sistema operativo. En Linux, como es mi caso, el valor por defecto suele ser 8 MB y puede consultarse o modificarse con el comando `ulimit -s`. Cuando la recursión excede este límite se produce un desbordamiento de pila (*stack overflow*).

Pseudocódigos de los Experimentos

Algorithm 1: Experimento 1 Orden de la Pila

Input: Ninguna

Output: Direcciones y orden de variables locales en la pila

```
1 Declarar  $a \leftarrow 10$ ,  $b \leftarrow 20$ ,  $c \leftarrow 30$ ,  $d \leftarrow 40$ 
2 foreach variable  $v \in \{a, b, c, d\}$  do
3   | Imprimir nombre, valor y dirección de memoria de  $v$ 
4 end
5 Calcular diferencias  $\Delta_{ab} \leftarrow \text{dir}(a) - \text{dir}(b)$ 
6 if  $\text{dir}(a) > \text{dir}(d)$  then
7   | Imprimir “La pila crece hacia direcciones bajas”
8 end
9 else
10  | Imprimir “La pila crece hacia direcciones altas”
11 end
```

Algorithm 2: Experimento 2 Stack Frames entre Funciones

```
1 Función FunciónHija( $param$ )
2   |  $local\_hija \leftarrow 999$ 
3   | Imprimir dirección de  $param$  y de  $local\_hija$ 
4 Función Experimento2()
5   |  $local\_padre \leftarrow 100$ 
6   | Imprimir dirección de  $local\_padre$ 
7   | FunciónHija( $local\_padre$ )
8   | Comparar direcciones: la hija vive más abajo que el padre (LIFO)
```

Algorithm 3: Experimento 3 — Alteración Indirecta de Valores

```
1 Función ModificarPorPuntero( $ptr$ )
2   |  $*ptr \leftarrow 777$ 
3   | Imprimir valor modificado
4 Función ModificarPorReferencia( $\&ref$ )
5   |  $ref \leftarrow 888$ 
6   | Imprimir valor modificado
7  $variable \leftarrow 42$ 
8 ModificarPorPuntero( $\&variable$ )           // modifica variable en el marco padre
9 ModificarPorReferencia( $variable$ )         // también modifica el marco padre
10 Imprimir valor final de  $variable$ 
```

Algorithm 4: Experimento 4 Límite de la Pila (Stack Overflow)

```
1 Función RecursiónProfunda(nivel, max)
2   Reservar arreglo local buffer[1024]           // ~1 KB por frame
3   if nivel mod 500 = 0 then
4     | Imprimir nivel y dirección de buffer
5   end
6   if nivel < max then
7     | RecursiónProfunda(nivel + 1, max)
8   end
9 RecursiónProfunda(1, 3000)
10 Imprimir si completó sin desbordamiento
```

Algorithm 5: Experimento 5 Paso por Valor vs. Paso por Referencia

```
1 Función PasoPorValor(x)
2   Imprimir dirección de x           // es una copia, dirección distinta
3   x ← 0
4 Función PasoPorReferencia(&x)
5   Imprimir dirección de x           // misma dirección que el original
6   x ← 0
7 original ← 555
8 PasoPorValor(original)
9 Verificar que original no cambió
10 PasoPorReferencia(original)
11 Verificar que original sí cambió
```

Resultados

En cuanto a los resultados esperados, en el experimento 1 las variables locales tienen direcciones separadas por 4 bytes (tamaño de `int`), y dependiendo de la arquitectura y el compilador, la pila puede crecer hacia direcciones bajas o altas. En el experimento 2, las variables de la función hija residen en direcciones más bajas que las del padre, confirmando el apilamiento LIFO. En el experimento 3, tanto el puntero como la referencia permiten modificar la variable del marco padre desde una función diferente, demostrando la alteración indirecta. En el experimento 4, con 3 000 niveles y aproximadamente 1 KB por frame (alrededor de 3 MB en total) la recursión se completa sin problemas, pero al incrementar los niveles o el buffer, eventualmente se produce un *segmentation fault* por desbordamiento. En el experimento 5, en paso por valor la dirección del parámetro difiere de la original porque es una copia, mientras que en paso por referencia ambas direcciones coinciden.

Compilación y Ejecución

Para la compilación y ejecución se recomienda compilar sin optimizaciones, ya que la bandera `-O0` evita que el compilador elimine variables locales, las almacene en registros o las reordene, lo cual alteraría los resultados de los experimentos [2]:

```
# Compilar sin optimizaciones para observar la pila real
g++ -O0 -o pila_funciones pila_funciones.cpp
# Ejecutar
./pila_funciones
# Consultar tamaño máximo de pila (en KB)
ulimit -s
# Cambiar tamaño de pila a 64 MB
ulimit -s 65536
```

Conclusión

Finalmente, los cinco experimentos permiten comprobar de forma práctica el funcionamiento interno de la pila de llamadas: desde el orden de las variables locales y la relación entre marcos de activación, hasta los mecanismos de paso de parámetros y los límites físicos de la pila. Entender estos conceptos es indispensable para diseñar algoritmos recursivos eficientes y para depurar problemas como el desbordamiento de pila, que son habituales en la práctica algorítmica.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Boston, MA: Pearson, 2015.
- [3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.